

Отчёт по практической работе №1

Группа ЕТ-442 Замятин Андрей Александрович

1 Задание

Требовалось написать на языке Python 3 программу, которая создает и обучает нейронную сеть для распознавания рукописных цифр с использованием метода градиентного спуска и базы данных MNIST. Работа выполнена по указаниям к практической работе 1.

2 Исходные данные

Сеть содержит три слоя. Входной слой принимает изображения размером 28 на 28 пикселей. Это дает 784 входа. Скрытый слой содержит 30 нейронов. Выходной слой содержит 10 нейронов по числу цифр от 0 до 9. Использованы язык Python версии 3.14.4 и библиотека NumPy ветки 2.4. Версия Python видна в строке состояния редактора на скриншоте в разделе 6. Признак версии NumPy виден на том же скриншоте в виде предупреждения про NumPy 2.4. Файл данных mnist.pkl.gz заранее загружен и находится в папке practical-1 рядом с программой, так как загрузчик открывает его по относительному имени.

3 Состав программы

Программа состоит из двух модулей. Модуль network.py описывает класс Network и функции активации. Модуль mnist_loader.py загружает файл mnist.pkl.gz и приводит наборы к виду, удобному для обучения. Набор обучения содержит 50000 пар входного изображения и ожидаемого вектора. Вектор ожидаемого ответа имеет длину 10 и содержит единицу в позиции верной цифры. Набор проверки содержит 10000 пар. Набор тестирования содержит 10000 пар. Такое разделение соответствует структуре базы MNIST.

4 Алгоритм

Для активации использована сигмоидальная функция. Она вычисляется через экспоненту из NumPy в векторизованной форме и применяется поэлементно. Ее производная выражена через значение самой функции по формуле $s'(z) = s(z)(1 - s(z))$, где $s(z)$ это сигмоида. В коде функция sigmoid_prime вызывает sigmoid(z) дважды, поэтому экспонента фактически вычисляется дважды. Код не сохраняет промежуточное значение сигмоиды для повторного использования.

Прямое распространение последовательно вычисляет потенциалы слоев как произведение матрицы весов на сигналы предыдущего слоя с добавлением смещения. Затем к потенциалам применяется активация. Одного вызова метода прямого распространения достаточно для получения ответа сети.

Обучение выполнено методом стохастического градиентного спуска. В начале каждой эпохи обучающая выборка перемешивается случайным образом. Затем она делится на малые подвыборки размером 10. Для каждой подвыборки вызывается один шаг обновления весов и смещений.

Градиенты для каждой пары из подвыборки вычисляются методом обратного распространения. Сначала выполняется прямой проход с запоминанием потенциалов и активаций всех слоев. Ошибка выходного слоя находится как разность выхода сети и ожидаемого вектора, умноженная на производную активации. Ошибка скрытых слоев находится через транспонированные матрицы весов со сдвигом по слоям назад. Затем для каждого слоя вычисляются градиенты по смещениям и по весам. Градиенты всех пар подвыборки суммируются.

Весы и смещения обновляются вычитанием среднего градиента, умноженного на скорость обучения 3.0. После завершения эпохи вызывается оценка точности на тестовом наборе. Ответом сети считается номер выходного нейрона с наибольшей активацией. Подсчитывается число верных ответов из 10000.

5 Запуск в используемой системе

В указаниях описан запуск через среду IDLE в Windows с ручной установкой рабочего каталога. Фактическая работа выполнена в редакторе VS Code запуском из терминала интерпретатором Python напрямую без IDLE.

Текущий каталог предварительно установлен в каталог с файлами программы и базой данных. Это важно, так как загрузчик открывает файл `mnist.pkl.gz` по относительному имени. Проверка показала 50000 пар в обучении и по 10000 пар в проверке и тестировании. Инициализация сети с топологией 784, 30, 10 прошла без ошибок.

Для запуска обучения использована команда из файла `task.txt`:

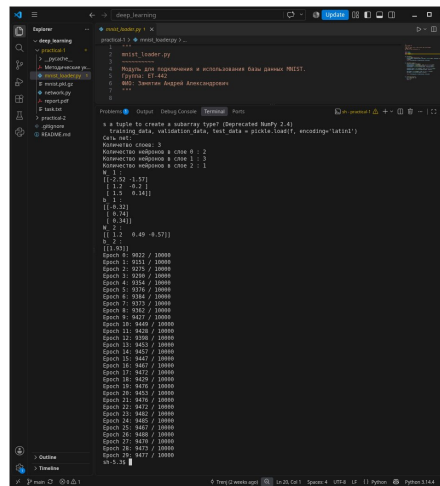
```
python3 -c "import mnist_loader; training_data, validation_data, test_data =
mnist_loader.load_data_wrapper(); import network; net = network.Network([784, 30, 10]);
net.SGD(training_data, 30, 10, 3.0, test_data=test_data)"
```

При подключении модуля `network` выполняется тело программы из методички и на экран выводится информация о тестовой сети 2, 3, 1. Затем создается сеть 784, 30, 10 и запускается обучение. Обучение занимает несколько минут. В ходе обучения на экран выводится строка для каждой эпохи с номером эпохи и числом верных ответов из 10000.

6 Результат

Обучение выполнено за 30 эпох. Вывод программы полностью сохранен на скриншоте ниже. На нулевой эпохе получено 9022 верных ответа из 10000. Далее точность в целом росла с колебаниями. На последней эпохе с номером 29 получено 9477 верных из 10000. Наилучший результат 9488 верных из 10000 зафиксирован на эпохе с номером 26. Такой ход подтверждает корректность загрузки данных и работы методов обучения и оценки.

Скриншот терминала с выводом обучения сети за 30 эпох



```
Training data, validation data, test data = pickle.load(), training_data_loader, validation_data_loader, test_data_loader

data_loader = DataLoader(training_data_loader)
validation_loader = DataLoader(validation_data_loader)
test_loader = DataLoader(test_data_loader)

# Create the model
model = nn.LSTM(28, 784, batch_first=True)

# Create the optimizer
optimizer = optim.Adam(model.parameters())

# Create the trainer
trainer = GradientDescentTrainer(model, data_loader, validation_loader, test_loader, optimizer)

# Train the model
trainer.train()

# Print the accuracy and loss for each epoch
for epoch in range(1, 26):
    accuracy, loss = trainer.evaluate()
    print(f'Epoch {epoch}: accuracy = {accuracy}, loss = {loss}')
```

7 Вывод

Собрана и обучена трехслойная сеть с топологией 784, 30, 10 для входа 28 на 28. Все методы класса Network реализованы по указаниям, включая прямое распространение, стохастический градиентный спуск, обновление мини пакета, обратное распространение, оценку точности и производную стоимости. Загрузка данных MNIST выполнена корректно с приведением размерностей к виду 784 на 1 для входов и 10 на 1 для ответов. Обучение сходится, точность на тестовом наборе растет по эпохам, что видно по скриншоту. Отличие от текста указаний состоит в том, что функция load_data_wrapper возвращает list(zip(...)) вместо zip. Это безвредно, так как метод SGD все равно преобразует выборки в списки, и обучение проходит без ошибок. Подключение модуля network дополнительно печатает параметры тестовой сети 2, 3, 1, как предусмотрено телом программы в указаниях.

Исходный код

Актуальная версия программы для данной практики хранится по адресу https://github.com/Trenj/deep_learning/tree/main/practical-1